**Homework #3: Fundamentals of high-performance methods for digital image and video processing using OpenCV**

**Warning:** You are expected to provide answers in your own words based on the discussion held in class. You will receive no credit for copying material from an AI chatbot, the internet, or if your discussion simply does not reflect the class discussion of what is expected. The use of AI or online resources is not prohibited. However, you need to use your own words, and your answers need to align with class discussions.

The first problem is worth 3 times as much as the second.

**Functional programming versus Object-oriented programming**

In object-oriented programming (OOP), inside a class, we store data in internal class variables. When a class method is called, the result depends on the results of previous calculations. To prove the correctness of OOP classes, you must show that you get the correct outputs for any possible input state and method applied to it.

Now, suppose that no internal or global variables are allowed in OOP. Then, to prove correctness, we only need to prove that we can generate the correct output from a given input. We often call this type of function a pure function.

Now, let us go one step further. Once you produce the output, you do not allow yourself to change it. We refer to this property as immutability.

One more step is to treat functions as first-class citizens. In this case, functions are treated as variables that you can pass around and execute in different parts of the code.

We define functional programming as a programming paradigm that uses pure functions, outputs are immutable, and functions are first-class citizens. For Python's implementation of functional programming concepts, refer to Python's official guide to functional programming.

**Multiple processes in Python**
In Python, we can use the multiprocessing package to achieve true parallelism on multiple cores. Each process runs the full Python interpreter as a child process. Refer to page 708 of Chapter 19 of *Fluent Python*, 2nd edition for details.

**Install OpenCV**
Refer to the installation instructions in Anaconda installation notes. If you want to install OpenCV in it's own environment, refer to OpenCV's official guide.

**Demos from Al Bovik**
During the discussion, I have used Al Bovik's demos that can be obtained from his Google drive link.

**Problem #1.**
The entire question refers to code and modifications to `gabor threads.py`. You will need to provide the modified code for credit and discuss your changes.

**1(a)**
The example provides an example with functional programming characteristics. Based on `gabor threads.py`, provide code samples that demonstrate:

i. Pure functions. do the outputs only depend on the inputs only (no use of internal memory variables)?

**Solution:**
A pure function is one whose output depends only on its inputs (or fixed constants inside the function) and does not rely on or modify internal or global state.

An example from `gabor_threads.py` is:

```
def build_filters():
filters = []
ksize = 31
for theta in np.arange(0, np.pi, np.pi / 16):
kern = cv.getGaborKernel((ksize, ksize), 4.0, theta,
10.0, 0.5, 0, ktype=cv.CV_32F)
kern /= 1.5 * kern.sum()
filters.append(kern)
return filters
```

This function is pure because:

- It does not use or modify global variables.
- It does not store persistent internal state.
- Its output is fully determined by fixed parameters inside the function.

Each call to `build_filters()` produces the same result and does not depend on previous executions, which satisfies the definition of a pure function.

ii. Immutability. Once a variable is assigned, does it stay fixed?

**Solution:**
Immutability means that once a value is assigned, it is not modified. Instead of changing the original data, a new result is produced.

An example from `gabor_threads.py` is:

```
def f(kern):
return cv.filter2D(img, cv.CV_8UC3, kern)
```

In this case, the function does not modify `img` or `kern`. It returns a new filtered image while leaving the original inputs unchanged. This demonstrates immutability because the existing data remains fixed after assignment, and the computation produces a new output rather than altering the inputs.

iii. Functions as first-class citizens. Do functions get passed around as variables? There are two examples of this usage in the code.

**Solution:**
Functions are considered first-class citizens when they can be treated like variables — meaning they can be defined inside other functions, assigned to a name, and passed as arguments to other functions.

The first example in `gabor_threads.py` is the nested function definition:

```
def process_threaded(img, filters, threadn=8):
accum = np.zeros_like(img)

def f(kern):
return cv.filter2D(img, cv.CV_8UC3, kern)
```

Here, the function `f` is defined inside another function and stored as a variable. This shows that functions can be created and handled like regular objects.

The second example is where this function is passed as an argument:

```
for fimg in pool.imap_unordered(f, filters):
np.maximum(accum, fimg, accum)
```

In this case, `f` is passed to `imap_unordered`, which executes it across multiple threads. This demonstrates that functions can be passed around and executed elsewhere in the program, satisfying the definition of functions as first-class citizens.

**Problem 1(b).**
In our use of Python for parallel processing, we are launching the Python interpreter with every process. Let the serial execution of a function on a single core be $t_{core}$. Let $n$ be the number of CPU cores. Answer the following:

i. Let the overhead of running the Python interpreter be $t_{interpreter}$. Let $t_{no\text{-}interpreter}$ refer to the execution time that does not involve the interpreter. We have:

$$t_{core} = t_{interpreter} + t_{no\text{-}interpreter}.$$

Assume that $t_{interpreter} > t_0$, which includes the minimum setup time to run the interpreter in a thread. Using $t_{interpreter}$ and $t_{no\text{-}interpreter}$, explain why the use of NumPy or OpenCV functions with high computational cost is recommended.

**Solution:**
If $t_{no\text{-}interpreter}$ is small, the interpreter overhead $t_{interpreter}$ dominates, limiting parallel speedup. Much of the time is then spent managing Python rather than computing.

Using NumPy or OpenCV increases $t_{no\text{-}interpreter}$ since the heavy work runs in optimized compiled code. This reduces the relative impact of $t_{interpreter}$ and makes parallelism more effective.

ii. What is the ideal execution time for $n$ processors? How close is your execution time to the ideal time? Explain.

**Solution:**
The ideal execution time with $n$ processors is

$$t_{\text{ideal}} = \frac{t_{core}}{n}.$$

This corresponds to a perfect $n\times$ speedup. In practice, the measured execution time is larger than $\frac{t_{core}}{n}$ due to interpreter overhead, thread management, and system-level costs. Therefore, the actual speedup is typically less than ideal.

**Problem 1(c).**
We want to generate DFT magnitude plots for the Gabor filters. To do this, create a unit impulse image and input it to the code. Your input image should be zero everywhere except for the central pixel that is 1. Use this image as input and apply each Gabor filter iteratively and then call the DFT magnitude plot function to generate magnitude plots.

This portion refers to the following code:

```
def build_filters():
filters = []
ksize = 31
for theta in np.arange(0, np.pi, np.pi / 16):
kern = cv.getGaborKernel((ksize, ksize), 4.0,
theta, 10.0, 0.5, 0,
ktype=cv.CV_32F)
kern /= 1.5 * kern.sum()
filters.append(kern)
return filters
```

**Solution:**
A unit impulse image (all zeros with the center pixel set to 1) was created and filtered with each Gabor kernel from `build_filters()`. The DFT of each filtered result was computed, the log-magnitude was taken, shifted to center the origin, normalized, and saved. Since the filter bank uses 16 orientations, this produced 16 DFT magnitude plots.

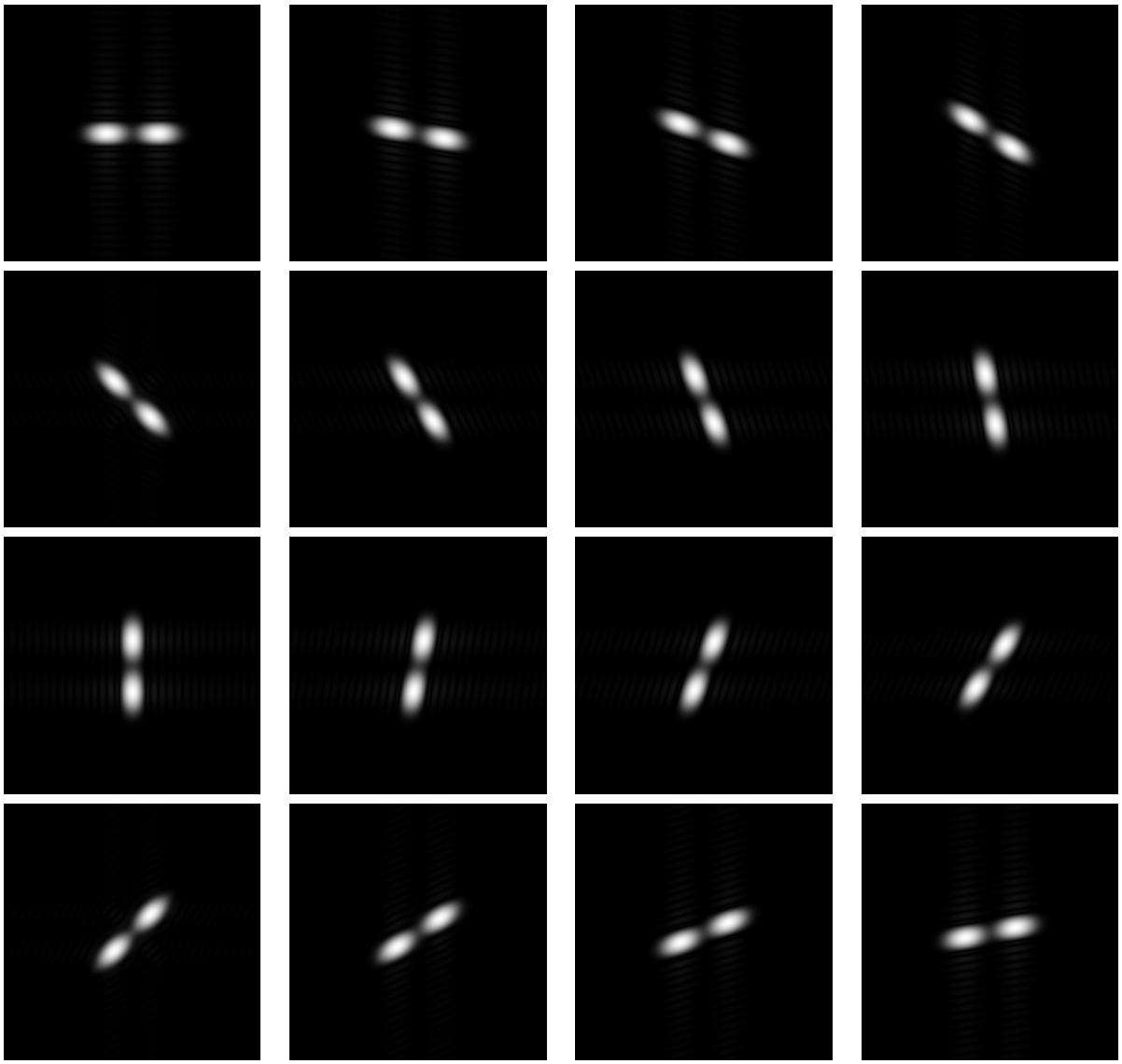

Figure 1: DFT magnitude plots of the 16 Gabor filters (varying orientation $\theta$).

**Problem 1(d).**
Explain the shape of the Gabor filter magnitude plots based on the parameters to `cv.getGaborKernel()`. Specifically, comment on the shape (i) ellipse or circle (why?), (ii) spread (ellipses have two spreads, circles have one), and (iii) central frequency location.

**Solution:**
**(i) Shape (ellipse vs. circle):** The magnitude response is generally *elliptical* because the Gabor kernel is anisotropic: it is a sinusoid modulated by a Gaussian envelope that is stretched differently along the two axes. The orientation parameter $\theta$ rotates this anisotropy, so the ellipse rotates as $\theta$ changes. A circular shape would require equal spread in all directions (isotropic Gaussian), which is not the case here.

**(ii) Spread:** The spread is different along two directions (major/minor axes of the ellipse). The parameters `sigma` and `gamma` control this: `sigma` sets the overall Gaussian width, and `gamma` (aspect ratio) makes the envelope narrower in one direction and wider in the other. This is why an ellipse has two distinct spreads.

**(iii) Central frequency location:** The magnitude is centered away from DC because the kernel contains a cosine with wavelength `lambd` (here $\lambda = 10$), which sets a nonzero spatial frequency. The peak locations appear as symmetric lobes about the origin, and their angle in the frequency plane matches the filter orientation $\theta$.

**Problem 1(e).**
Explain what the `process()` function is doing. Sketch an example that shows what the `maximum()` function is doing.

```
def process(img, filters):
accum = np.zeros_like(img)
for kern in filters:
fimg = cv.filter2D(img, cv.CV_8UC3, kern)
np.maximum(accum, fimg, accum)
return accum
```

**Solution:**
The function `process()` applies each Gabor filter to the image and keeps the strongest response at every pixel. It initializes `accum` to zeros and, for each filtered image `fimg`, updates

$$\text{accum}(x, y) = \max(\text{accum}(x, y), \text{fimg}(x, y)).$$

A sketch of the `maximum()` operation is shown below:

$$\text{accum (before)} \qquad \text{fimg}$$

$$\begin{bmatrix} 1 & 5 \\ 3 & 2 \end{bmatrix} \qquad \begin{bmatrix} 4 & 2 \\ 1 & 6 \end{bmatrix}$$

$$\Downarrow \quad \text{elementwise max}$$

$$\text{accum (after)} = \begin{bmatrix} 4 & 5 \\ 3 & 6 \end{bmatrix}$$

Each entry in the result is the larger value from the corresponding positions. After all filters are processed, the final image contains the maximum response across all filter orientations at each pixel.

**Problem 1(f).**
For `process_threaded()`, answer the following questions:

i. Why are we using `imap_unordered(f, filters)` here?

ii. How is `imap_unordered(f, filters)` executed?

iii. In terms of execution, what are the differences between `cv.filter2D(img, cv.CV_8UC3, kern)` and `pool.imap_unordered(f, filters)`?

**Solution:**

i. `imap_unordered(f, filters)` is used so that filtered results are returned as soon as any thread finishes, instead of waiting for them in the original order. This reduces idle time and improves throughput when different filters take slightly different amounts of time.

ii. The thread pool schedules calls to `f(kern)` across the available worker threads. Each element of `filters` is submitted as a task, and `imap_unordered` yields each completed `fimg` as soon as it finishes, in whatever order tasks complete.

iii. `cv.filter2D(img, ..., kern)` is a single, direct (serial) convolution call that runs once for one kernel in the current thread. In contrast, `pool.imap_unordered(f, filters)` runs many `cv.filter2D` calls concurrently by distributing different kernels to different threads, and returns a stream of results as they complete, with added overhead from task scheduling and thread management.

**Problem 1(g).**
For this problem, retrieve the number of CPU cores using:

```
threadn = cv.getNumberOfCPUs()
```

Answer the following:

i. Compute the times and speedup factors using 0.5, 1.0, 2.0 times the number of available CPUs.

ii. Determine the optimal value for the pool size.

iii. Did the optimal speedup meet your expectations? Explain why or why not.

**Solution:**
Using `threadn = cv.getNumberOfCPUs()`, my system reports $n = 10$ CPUs. The measured single-thread runtime was $T_1 = 0.2689\,\text{s}$. I then tested pool sizes $k \in \{0.5n,\ n,\ 2n\} = \{5, 10, 20\}$ and computed speedup as $S_k = T_1/T_k$.

For $k = 5$: $T_5 = 0.1086\,\text{s}$, $S_5 = 2.475$.
For $k = 10$: $T_{10} = 0.0867\,\text{s}$, $S_{10} = 3.100$.
For $k = 20$: $T_{20} = 0.0623\,\text{s}$, $S_{20} = 4.314$.

The optimal pool size (lowest runtime / highest speedup) among these tests was $k = 20$ threads. The speedup is below the ideal linear speedup due to overhead and resource limits (thread scheduling, memory/cache effects, and combining results), but increasing the pool size to $2n$ still improved performance for this workload.

**Problem 1(h).**
Double the number of Gabor filters used for processing. Does the speedup improve? Explain.

**Solution:**
I doubled the number of Gabor filters from 16 to 32 and repeated the timing/speedup experiment using $n = 10$ CPUs and pool sizes $\{5, 10, 20\}$.

For 16 filters, $T_1 = 0.2440\,\text{s}$ and the best result occurred at 20 threads with $T_{20} = 0.0754\,\text{s}$, giving a speedup of $S_{20} = 3.234$.

For 32 filters, $T_1 = 0.4752\,\text{s}$ and the best result also occurred at 20 threads with $T_{20} = 0.1049\,\text{s}$, giving a speedup of $S_{20} = 4.529$.

Therefore, the speedup *did* improve when doubling the number of filters (from 3.234 to 4.529). This makes sense because increasing the workload increases the amount of parallelizable computation per run, so the fixed overhead of thread scheduling becomes a smaller fraction of the total runtime, allowing the thread pool to be utilized more efficiently.

**Problem #2**
This problem refers to `video_threaded.py`.
**2(a)**
Answer the following parts.

i. Suppose that the input video is at 30 frames-per-second. What is the minimum frame interval for streaming real-time video?

ii. Suppose that you are designing an emergency notification system. What do you prefer? Low latency or low frame interval processing? Why?

iii. What does latency represent?

iv. What does frame interval represent?

v. Give an example of high-latency and low frame-interval and explain how this occurs.

**Solution:**

i. At 30 frames-per-second, the minimum frame interval for real-time streaming is

$$\Delta t = \frac{1}{30} \text{ s} \approx 0.0333 \text{ s} = 33.3 \text{ ms}.$$

ii. For an emergency notification system, I prefer **low latency** because the most important requirement is delivering the newest information as quickly as possible, even if fewer frames are shown.

iii. **Latency** is the time delay from when a frame is captured to when the processed result is displayed (end-to-end delay).

iv. **Frame interval** is the time between consecutive displayed frames (inverse of the displayed FPS).

v. **High-latency and low frame-interval example:** Suppose a system captures and displays frames frequently (small frame interval), but each frame is queued behind many pending processing tasks. The display stays smooth, but what you see is "old" (large capture-to-display delay), so latency is high even though the frame interval is low.

**2(b)** For this problem, we note that the number of CPU cores is retrieved using:

```
threadn = cv.getNumberOfCPUs()
```

Answer the following:

i. Compute the times and speedup factors using 0.5, 1.0, 2.0 times the number of available CPUs.

ii. Determine the optimal speedup for the pool size.

iii. Did the optimal speedup meet your expectations? Explain why or why not.

**Solution:**
Using `threadn = cv.getNumberOfCPUs()`, I tested pool sizes $k \in \{0.5n,\ n,\ 2n\}$ and recorded the steady-state frame interval. The single-thread baseline (threaded = False) was approximately

$$T_{\text{single}} = 93.5\,\text{ms}.$$

The measured multithreaded frame intervals were:

$$T_{0.5n} = 39.6\,\text{ms}, \quad T_n = 26.3\,\text{ms}, \quad T_{2n} = 22.9\,\text{ms}.$$

Speedup was computed as

$$S_k = \frac{T_{\text{single}}}{T_k}.$$

$$S_{0.5n} = 2.36, \quad S_n = 3.56, \quad S_{2n} = 4.08.$$

The optimal performance occurred at $k = 2n$, giving approximately a $4\times$ speedup. The speedup is below the ideal linear scaling due to thread management overhead, synchronization, and hardware limits.

**2(c)** Why are we using `apply_async()`?

**Solution:**
We use `apply_async()` to submit frame-processing work to the thread pool *without blocking* the main capture/display loop. This lets the program keep reading new frames while older frames are still being processed, creating a pipeline that improves throughput and produces smoother playback. If we used a blocking call, the program would have to wait for each processed frame before capturing the next one, reducing performance and making the video less smooth.

**2(d)** Why are we using `deque` for retrieving the results? What would happen if we didn't?
**Solution:**
We use `deque` to store pending processing tasks in a FIFO (first-in, first-out) structure. This allows efficient appending of new tasks to the right and popping completed tasks from the left with constant time complexity. In the video pipeline, frames are processed asynchronously, and `deque` lets us retrieve completed tasks in order while maintaining smooth playback.

If we did not use `deque`, managing pending tasks would be less efficient. For example, using a standard list would make popping from the front costly (requiring shifting elements), which could degrade performance. Without an ordered structure to track pending tasks, frames might be displayed out of order or the pipeline could stall, increasing latency and reducing smoothness.

**2(e)** Increase the processing time by repeating the median blur two more times. Does the speedup improve? Explain.

**Solution:**

I increased the per-frame processing time by adding two more calls to `cv.medianBlur(frame, 19)` (total of 4 blurs per frame). Using the on-screen steady-state frame interval:

$$T_{\text{single}} = 178.7\,\text{ms}, \qquad T_{\text{threaded}} = 37.0\,\text{ms}.$$

The speedup is

$$S = \frac{T_{\text{single}}}{T_{\text{threaded}}} = \frac{178.7}{37.0} = 4.83.$$

Yes, the speedup improves compared to the lighter workload because increasing the computation per frame makes the fixed overhead (task scheduling, queueing, and thread management) a smaller fraction of the total time, so the thread pool is utilized more effectively.

```
'''
Code to run main script
Usage:
nguyen-scott-ece-533-hw-3-code.py
'''
import os
import math
import time
import numpy as np
import cv2 as cv

from gabor_threads import build_filters, process, process_threaded
from dft import shift_dft


out_dir = "gabor_dft_outputs"
os.makedirs(out_dir, exist_ok=True)

size = 256
impulse = np.zeros((size, size), dtype=np.float64)
impulse[size // 2, size // 2] = 1.0

filters = build_filters()

for i, kern in enumerate(filters):
filtered = cv.filter2D(impulse, cv.CV_64F, kern)

h, w = filtered.shape[:2]
dft_M = cv.getOptimalDFTSize(w)
dft_N = cv.getOptimalDFTSize(h)

dft_A = np.zeros((dft_N, dft_M, 2), dtype=np.float64)
dft_A[:h, :w, 0] = filtered

cv.dft(dft_A, dst=dft_A, nonzeroRows=h)

re, im = cv.split(dft_A)
magnitude = cv.sqrt(re ** 2.0 + im ** 2.0)
log_spectrum = cv.log(1.0 + magnitude)

shift_dft(log_spectrum, log_spectrum)
cv.normalize(log_spectrum, log_spectrum, 0.0, 1.0, cv.NORM_MINMAX)

save_path = os.path.join(out_dir, f"gabor_dft_{i:02d}.png")
cv.imwrite(save_path, (log_spectrum * 255).astype(np.uint8))

print(f"Saved {len(filters)} DFT magnitude images to: {out_dir}")
```

```
cv.destroyAllWindows()


img = cv.imread("baboon.jpg")
if img is None:
raise ValueError("Failed to load baboon.jpg (put it in the same folder).")

n = cv.getNumberOfCPUs()
print(f"Number of CPUs: {n}")

test_threads = [
max(1, int(math.floor(0.5 * n))),
max(1, int(n)),
max(1, int(2 * n)),
]


start = time.time()
_ = process(img, filters)
T1 = time.time() - start
print(f"Single-thread time (16 filters): {T1:.4f} s")

results = {}
for k in test_threads:
start = time.time()
_ = process_threaded(img, filters, threadn=k)
Tk = time.time() - start
Sk = T1 / Tk
results[k] = (Tk, Sk)
print(f"Threads: {k:2d} | Time: {Tk:.4f} s | Speedup: {Sk:.3f}")

best_k = min(results, key=lambda kk: results[kk][0])
print(f"Best pool size (16 filters): {best_k} threads")


filters2 = filters + filters

start = time.time()
_ = process(img, filters2)
T1_2 = time.time() - start
print(f"\nSingle-thread time (32 filters): {T1_2:.4f} s")

results2 = {}
for k in test_threads:
start = time.time()
_ = process_threaded(img, filters2, threadn=k)
Tk = time.time() - start
Sk = T1_2 / Tk
```

```
results2[k] = (Tk, Sk)
print(f"Threads: {k:2d} | Time: {Tk:.4f} s | Speedup: {Sk:.3f}")

best_k2 = min(results2, key=lambda kk: results2[kk][0])
print(f"Best pool size (32 filters): {best_k2} threads")
```

```
#!/usr/bin/env python
'''
Multithreaded video processing sample.
Usage:
video_threaded.py {<video device number>|<video file name>}

Shows how python threading capabilities can be used
to organize parallel captured frame processing pipeline
for smoother playback.

Keyboard shortcuts:
ESC - exit
space - switch between multi and single threaded processing
'''

# Python 2/3 compatibility
from __future__ import print_function

import numpy as np
import cv2 as cv
from multiprocessing.pool import ThreadPool
from collections import deque

from common import clock, draw_str, StatValue
import video


class DummyTask:
def __init__(self, data):
self.data = data

def ready(self):
return True

def get(self):
return self.data


def main():
import sys
try:
fn = sys.argv[1]
except:
fn = 0

cap = video.create_capture(fn)

def process_frame(frame, t0):
```

```
frame = cv.medianBlur(frame, 19)
frame = cv.medianBlur(frame, 19)
frame = cv.medianBlur(frame, 19)
frame = cv.medianBlur(frame, 19)
return frame, t0

n = cv.getNumberOfCPUs()
threadn = int(2 * n)
pool = ThreadPool(processes=threadn)
pending = deque()

threaded_mode = True

latency = StatValue()
frame_interval = StatValue()
last_frame_time = clock()

while True:
while len(pending) > 0 and pending[0].ready():
res, t0 = pending.popleft().get()
latency.update(clock() - t0)

draw_str(res, (20, 20), "threaded : " + str(threaded_mode))
draw_str(res, (20, 40), "latency : %.1f ms" % (latency.value * 1000))
draw_str(res, (20, 60), "frame interval : %.1f ms" % (frame_interval.value * 1000))

cv.imshow('threaded video', res)

if len(pending) < threadn:
_ret, frame = cap.read()
t = clock()
frame_interval.update(t - last_frame_time)
last_frame_time = t

if threaded_mode:
task = pool.apply_async(process_frame, (frame.copy(), t))
else:
task = DummyTask(process_frame(frame, t))

pending.append(task)

ch = cv.waitKey(1)
if ch == ord(' '):
threaded_mode = not threaded_mode
if ch == 27:
break

print('Done')
```

```
if __name__ == '__main__':
print(__doc__)
main()
cv.destroyAllWindows()
```

```
#!/usr/bin/env python
'''
sample for discrete fourier transform (dft)

USAGE:
dft.py <image_file>
'''

from __future__ import print_function
import numpy as np
import cv2 as cv
import sys


def shift_dft(src, dst=None):
'''
Rearrange the quadrants of Fourier image so that the origin is at
the image center. Swaps quadrant 1 with 3, and 2 with 4.
src and dst arrays must be equal size & type
'''
if dst is None:
dst = np.empty(src.shape, src.dtype)
elif src.shape != dst.shape:
raise ValueError("src and dst must have equal sizes")
elif src.dtype != dst.dtype:
raise TypeError("src and dst must have equal types")

if src is dst:
ret = np.empty(src.shape, src.dtype)
else:
ret = dst

h, w = src.shape[:2]
cx1 = cx2 = w // 2
cy1 = cy2 = h // 2

if w % 2 != 0:
cx2 += 1
if h % 2 != 0:
cy2 += 1

ret[h-cy1:, w-cx1:] = src[0:cy1, 0:cx1]
ret[0:cy2, 0:cx2] = src[h-cy2:, w-cx2:]
ret[0:cy2, w-cx2:] = src[h-cy2:, 0:cx2]
ret[h-cy1:, 0:cx1] = src[0:cy1, w-cx1:]

if src is dst:
dst[:, :] = ret
```

```
return dst


def main():
if len(sys.argv) > 1:
fname = sys.argv[1]
else:
fname = 'baboon.jpg'

print("usage: python dft.py <image_file>")

im = cv.imread(cv.samples.findFile(fname))
im = cv.cvtColor(im, cv.COLOR_BGR2GRAY)

h, w = im.shape[:2]
realInput = im.astype(np.float64)

dft_M = cv.getOptimalDFTSize(w)
dft_N = cv.getOptimalDFTSize(h)

dft_A = np.zeros((dft_N, dft_M, 2), dtype=np.float64)
dft_A[:h, :w, 0] = realInput

cv.dft(dft_A, dst=dft_A, nonzeroRows=h)

cv.imshow("win", im)

image_Re, image_Im = cv.split(dft_A)
magnitude = cv.sqrt(image_Re ** 2.0 + image_Im ** 2.0)
log_spectrum = cv.log(1.0 + magnitude)

shift_dft(log_spectrum, log_spectrum)

cv.normalize(log_spectrum, log_spectrum, 0.0, 1.0, cv.NORM_MINMAX)
cv.imshow("magnitude", log_spectrum)

cv.waitKey(0)
print('Done')


if __name__ == '__main__':
print(__doc__)
main()
cv.destroyAllWindows()
```

```
#!/usr/bin/env python
'''
gabor_threads.py
=========
Sample demonstrates:
- use of multiple Gabor filter convolutions to get Fractalius-like image effect (http://www.r
- use of python threading to accelerate the computation

Usage
-----
gabor_threads.py [image filename]
'''

# Python 2/3 compatibility
from __future__ import print_function
import numpy as np
import cv2 as cv
from multiprocessing.pool import ThreadPool

def build_filters():
filters = []
ksize = 31
for theta in np.arange(0, np.pi, np.pi / 16):
kern = cv.getGaborKernel((ksize, ksize), 4.0, theta, 10.0, 0.5, 0, ktype=cv.CV_32F)
kern /= 1.5 * kern.sum()
filters.append(kern)
return filters

def process(img, filters):
accum = np.zeros_like(img)
for kern in filters:
fimg = cv.filter2D(img, cv.CV_8UC3, kern)
np.maximum(accum, fimg, accum)
return accum

def process_threaded(img, filters, threadn=8):
accum = np.zeros_like(img)
def f(kern):
return cv.filter2D(img, cv.CV_8UC3, kern)
pool = ThreadPool(processes=threadn)
for fimg in pool.imap_unordered(f, filters):
np.maximum(accum, fimg, accum)
return accum

def main():
import sys
from common import Timer
```

```
try:
img_fn = sys.argv[1]
except:
img_fn = 'baboon.jpg'

img = cv.imread(img_fn)
if img is None:
print('Failed to load image file:', img_fn)
sys.exit(1)

filters = build_filters()

with Timer('running single-threaded'):
res1 = process(img, filters)

with Timer('running multi-threaded'):
res2 = process_threaded(img, filters)

print('res1 == res2 :', (res1 == res2).all())
cv.imshow('img', img)
cv.imshow('result', res2)
cv.waitKey()

print('Done')

if __name__ == '__main__':
print(__doc__)
main()
cv.destroyAllWindows()
```